

GITHUB ACTION

Step 1: Create a GitHub Repository

- Go to GitHub and log in.
- Click on New Repository.
- Enter a name (e.g., github-actions-demo).
- Select Public or Private, and click Create Repository.

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

Owner * Jaydeep-007 / Repository name * github-actions-demo
github-actions-demo is available.

Great repository names are short and memorable. Need inspiration? How about [crispy-octo-happiness](#) ?

Description (optional)

☒ Public
Anyone on the internet can see this repository. You choose who can commit.

☐ Private
You choose who can see and commit to this repository.

Initialize this repository with:

☒ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs](#).

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. [Learn more about ignoring files](#).

Choose a license

License: MIT License

A license tells others what they can and can't do with your code. [Learn more about licenses](#).

This will set `main` as the default branch. Change the default name in your [settings](#).

You are creating a public repository in your personal account.

Create repository

Jaydeep-007 / github-actions-demo

Search Type / to search

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

github-actions-demo Public

Pin Unwatch 1

main 1 Branch 0 Tags

Go to file t

Add file

<> Code

Jaydeep-007 Initial commit ea9d108 · now 1 Commit

LICENSE	Initial commit	now
README.md	Initial commit	now

README MIT license

github-actions-demo

Step 2: You need the host to deploy and run the GitHub Runner which is needed to execute the jobs of our workflow. So, for that we gonna create a new VM for the same.

Create an Azure Virtual Machine (VM)

- Go to Azure Portal → Virtual Machines → Click Create → Virtual Machine.
- Choose Basic Configuration:

[Home](#) > [jaydeep-rg](#) > [Marketplace](#) >

Virtual machine ✕ ...
Microsoft



Virtual machine [Add to Favorites](#)

Microsoft | Azure Service

★ 4.5 (207 ratings)

Plan

Virtual machine

Create

[Overview](#) [Plans](#) [Usage Information + Support](#) [Ratings + Reviews](#)

With support for Linux, Windows Server, SQL Server, Oracle, IBM, and SAP, Azure Virtual Machines give you the flexibility of virtualization for a wide range of computing solutions—development and testing, running applications, and extending your datacenter. It's the freedom of open-source software configured the way you need it. It's as if it was another rack in your datacenter, giving you the power to deploy an application in seconds instead of weeks.

Azure Virtual Machines support IBM, Oracle, Red Hat, SAP, SQL Server, Linux, and Windows Server

More products from Microsoft [See All](#)



Active Directory Health Check

Microsoft

Azure Service

Assess the risk and health of Active Directory environments.

Create



AD Replication Status

Microsoft

Azure Service

Identify Active Directory replication issues in your environment.

Create



Device Update for IoT Hub

Microsoft

Azure Service

Securely and Reliably update your devices with Device Update for IoT Hub.

Create



Front Door and CDN profiles

Microsoft

Azure Service

Azure Front Door and CDN profiles is security led, modern cloud CDN that provides static and dynamic content acceleration, global load balancing and enhanced security for your apps.

Create



[Home](#) > [jaydeep-rg](#) > [Marketplace](#) > [Virtual machine](#) >

Create a virtual machine ...

[Help me create a low cost VM](#) [Help me create a VM optimized for high availability](#) [Help me choose the right VM size for my workload](#)

[Basics](#) [Disks](#) [Networking](#) [Management](#) [Monitoring](#) [Advanced](#) [Tags](#) [Review + create](#)

Create a virtual machine that runs Linux or Windows. Select an image from Azure marketplace or use your own customized image. Complete the Basics tab then Review + create to provision a virtual machine with default parameters or review each tab for full customization. [Learn more](#)

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Resource group * [Create new](#)

Instance details

Virtual machine name *

Region *

Availability options

Zone options ☒ Self-selected zone
Choose up to 3 availability zones, one VM per zone
☐ Azure-selected zone (Preview)
Let Azure assign the best zone for your needs

Availability zone *

☒ You can now select multiple zones. Selecting multiple zones will create one VM

[< Previous](#)

[Next : Disks >](#)

[Review + create](#)

[Home](#) > [jaydeep-rg](#) > [Marketplace](#) > [Virtual machine](#) >

Create a virtual machine ...

[Help me create a low cost VM](#) [Help me create a VM optimized for high availability](#) [Help me choose the right VM size for my workload](#)

Availability zone *
☒ You can now select multiple zones. Selecting multiple zones will create one VM per zone. [Learn more](#)

Security type [Configure security features](#)

Image * [See all images](#) [Configure VM generation](#)

VM architecture ☐ Arm64 ☒ x64

Run with Azure Spot discount ☐

Size * [See all sizes](#)

Enable Hibernation ☐
Learn more"/>

Administrator account

Authentication type ☐ SSH public key

[< Previous](#)

[Next : Disks >](#)

[Review + create](#)

Create a virtual machine ...

 [Help me create a low cost VM](#) [Help me create a VM optimized for high availability](#) [Help me choose the right VM size for my workload](#)

 Hibernate does not currently support Trusted launch and Confidential virtual machines for Linux images. [Learn more](#)

Administrator account

Authentication type 

☐ SSH public key
☒ Password

Username * 



Password *



Confirm password *



Inbound port rules

Select which virtual machine network ports are accessible from the public internet. You can specify more limited or granular network access on the Networking tab.

Public inbound ports * 

☐ None
☒ Allow selected ports

Select inbound ports *



 This will allow all IP addresses to access your virtual machine. This is only recommended for testing. Use the Advanced controls in the Networking tab to create rules to limit inbound traffic to known IP addresses.

[< Previous](#)

[Next : Disks >](#)

[Review + create](#)

- Click Review + Create → Create.

Connect to the VM

Click on the VM instance in azure and there in the side navbar you can see connect option. So, using that option to connect with your VM.

Step 3: Set Up a GitHub Self-Hosted Runner

By default, GitHub provides free hosted runners, but here we'll set up a self-hosted runner on your local machine.

1. Navigate to Runner Settings

- Go to GitHub Repository → Settings → Actions → Runners.
- Click New self-hosted runner.

The screenshot shows the GitHub Actions Runners configuration page. The left sidebar has a menu with options like General, Access, Collaborators, Moderation options, Code and automation, Branches, Tags, Rules, Actions, Runners (selected), Webhooks, Environments, Codespaces, Pages, and Security. The main content area is titled 'Runners' and includes a 'New self-hosted runner' button. A message states 'There are no runners configured' with a link to 'Learn more about using runners'. Below this, the 'Add new self-hosted runner' section is shown, with options for 'Runner image' (macOS, Linux, Windows) and 'Architecture' (x64). A 'Download' section contains the following commands:

```
# Create a folder
$ mkdir actions-runner && cd actions-runner

# Download the latest runner package
$ curl -o actions-runner-linux-x64-2.322.0.tar.gz -L
https://github.com/actions/runner/releases/download/v2.322.0/actions-runner-linux-x64-2.322.0.tar.gz

# Optional: Validate the hash
```

2. Choose the Operating System

- Select your OS (Windows, macOS, or Linux).

3. Download and Configure the Runner

To download and configure the GitHub Runner. First, you have to connect with the VM that we created previously and once the connection is successful then you can execute the commands which are visible on the runner creation page.

Follow the provided commands.

```
mkdir actions-runner && cd actions-runner
curl -o actions-runner-linux-x64.tar.gz -L
https://github.com/actions/runner/releases/download/v2.308.0/actions-runner-linux-
x64-2.308.0.tar.gz
tar xzf ./actions-runner-linux-x64.tar.gz
```

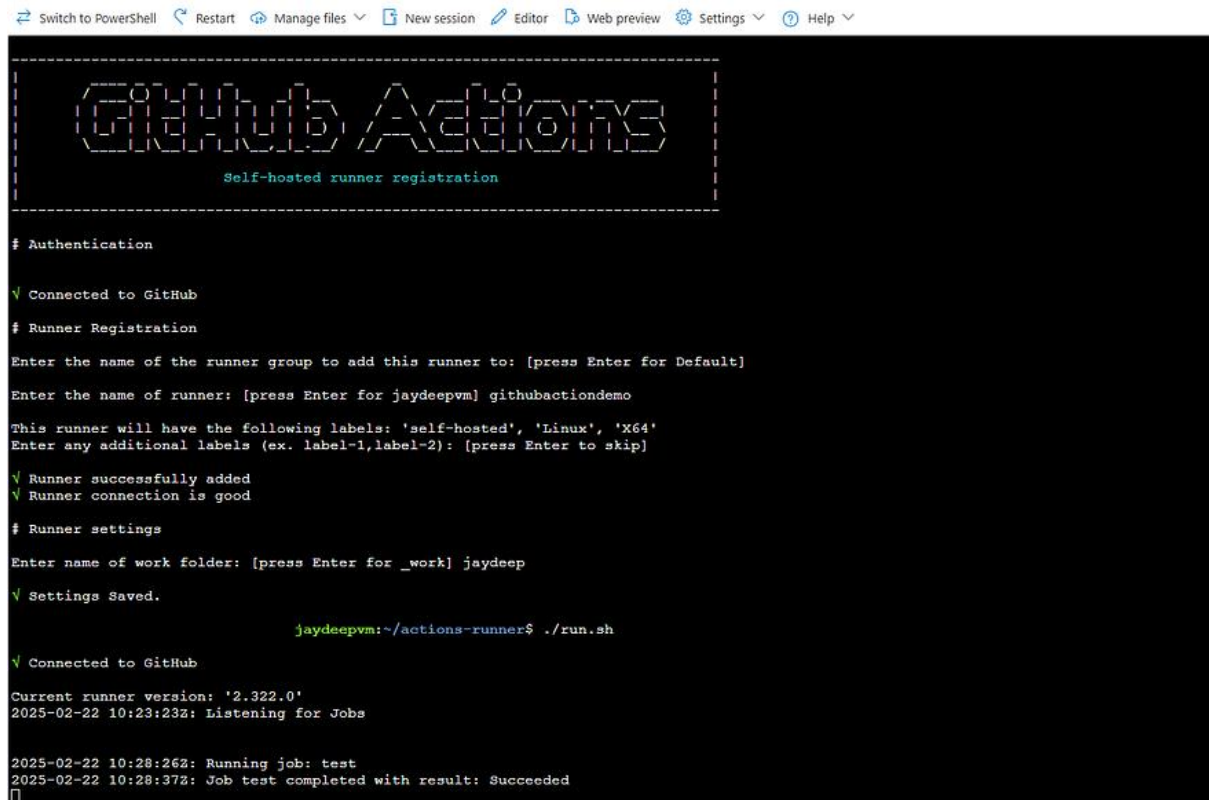
4. Configure the Runner

Use the provided command with your repository's token:

```
./config.sh --url https://github.com/your-username/github-actions-demo --token YOUR_TOKEN
```

5. Start the Runner

```
./run.sh
```



```
Switch to PowerShell Restart Manage files New session Editor Web preview Settings Help

GitHub Actions
Self-hosted runner registration

# Authentication
✓ Connected to GitHub

# Runner Registration
Enter the name of the runner group to add this runner to: [press Enter for Default]
Enter the name of runner: [press Enter for jaydeepvm] githubactiondemo
This runner will have the following labels: 'self-hosted', 'Linux', 'X64'
Enter any additional labels (ex. label-1,label-2): [press Enter to skip]
✓ Runner successfully added
✓ Runner connection is good

# Runner settings
Enter name of work folder: [press Enter for _work] jaydeep
✓ Settings Saved.

jaydeepvm:~/actions-runner$ ./run.sh

✓ Connected to GitHub
Current runner version: '2.322.0'
2025-02-22 10:23:23Z: Listening for Jobs

2025-02-22 10:28:26Z: Running job: test
2025-02-22 10:28:37Z: Job test completed with result: Succeeded
```

Your self-hosted runner is now active and ready!

Step 4: Set Up GitHub Actions Workflow

In your GitHub repository, create a file:

```
.github/workflows/test-runner.yml
```

Add this simple workflow to verify the runner:

```
name: Test Self-Hosted Runner
on: [push]
jobs:
  test:
    runs-on: self-hosted
    steps:
      - name: Print Hello
        run: echo "Hello from Azure VM Runner!"
```

Push a change to your repository and check the Actions tab in GitHub.

You should see the job running on your Azure VM!

Note: You might get some errors initially in the workflow due unavailability of node js and few other things. So, please installed that on by one on your VM.

This screenshot shows the 'All workflows' page in the GitHub Actions interface. The left sidebar contains a list of workflow files: 'Test Self-Hosted Runner', 'Caches', 'Attestations', 'Runners', 'Usage metrics', and 'Performance metrics'. The main area displays a single workflow run for 'Create test-runner.yml' on the 'main' branch, triggered by a push event. The run status is 'Success' and it completed 1 minute ago, taking 18 seconds.

This screenshot shows the 'Summary' page for the workflow run 'Create test-runner.yml #1'. The left sidebar lists 'Jobs' (with 'test' selected), 'Run details', 'Usage', and 'Workflow file'. The main content area shows the workflow was triggered by a push 1 minute ago and completed successfully in 18 seconds. Below this, the 'test-runner.yml' file content is displayed, showing a single job named 'test' that ran for 6 seconds.

This screenshot shows the 'Logs' page for the 'test' job. The left sidebar shows 'Jobs' with 'test' selected, and 'Run details', 'Usage', and 'Workflow file'. The main area displays the log output for the job, which succeeded 1 minute ago. The log shows the following steps: 'Set up job' (0s), 'Print Hello' (1s), and 'Complete job' (0s). The 'Print Hello' step is expanded, showing two lines of output: 'Run echo "Hello from Azure VM Runner!"' and 'Hello from Azure VM Runner!'. The second line is highlighted with a red box.